CS0011 FINAL PROJECT | APPLICATION PROPOSAL

WalkIt

Pedestrian Hazard Reporting and Walking Route Awareness

Community-powered navigation for safer and more accessible walking.

Application name	WalkIt
Category	Community concerns, pedestrian safety, and accessibility
Platform	Android application built in Android Studio using Kotlin
Database	Local SQLite database using SQLiteOpenHelper
Team	Arias, Sam Denielle Coo, Helijah Sephreen Escara, Angel John Euginio, Daniel Kenneth Salvador, John Carlo
Submission deadline	October 31, 2026
Proposal status	For instructor review and approval; implementation has not started

1. Executive Summary

WalkIt is a user-based Android application that helps people make better-informed walking decisions. Contributors record sidewalk hazards and accessibility barriers against named pathway segments, and other users consult those records when comparing predefined walking routes. Each contributor has a profile, a personal report history, saved routes, route preferences, badges, and a place on a local leaderboard.

This proposal defines the CS0011 prototype: an offline, single-installation app focused on one instructor-approved campus or neighborhood. It deliberately narrows the broader WalkIt product vision (live maps, cloud accounts, government dashboard) to a manageable, database-driven scope that can be designed, built, and tested by a five-member team in four two-week sprints.

Product goal: Deliver a demonstrable mobile app in which a user can maintain a personal profile, manage owned hazard reports, compare and save predefined routes, earn badges, and view a local contribution leaderboard within an agreed test area.

2. Background and Problem Statement

Pedestrians regularly encounter damaged sidewalks, blocked pathways, uneven surfaces, and accessibility barriers. Information about these conditions is usually scattered or unrecorded, so a person choosing a path may know its distance but little about problems along it. People with mobility-related needs are affected most, because a single barrier can make a route unusable.

WalkIt addresses this as an information-management problem: it provides a structured way to record concerns and to consult them when reviewing walking options.

> **Important limitation:** WalkIt does not certify that a route is safe or accessible, and it does not replace on-site judgment. Recommendations depend on the reports available, which may be limited or outdated.

Intended users

- **Students and daily commuters** who regularly walk through the selected area.
- **Pedestrians and visitors** who want to inspect recorded conditions before choosing a path.
- **People with mobility-related accessibility needs** who want to identify reported barriers along available routes.
- **A student or community volunteer** maintaining the prototype's local records: adding observations, correcting information, and marking issues as resolved.

3. Objectives

General objective

To design, develop, test, and present a user-based Kotlin Android application that uses SQLite to manage pedestrian hazard reports, personalize walking information, and recognize participation through profiles, badges, and a local contribution leaderboard.

Specific objectives

1. Implement complete create, view, update, and delete operations for hazard reports, with records retained after the app is closed and reopened.
2. Develop eight planned Activities with explicit Intent-based transfer of report, route, and public-profile identifiers and selected preferences.

3. Validate required inputs and provide clear success, error, empty-state, and confirmation messages.
4. Implement search, filtering, sorting, and predefined-route comparison as meaningful features beyond CRUD.
5. Apply Android resource organization and accessible interface practices, including readable text, labeled controls, and status indicators that do not rely on color alone.
6. Execute and document at least 12 test cases, record at least three actual defects and their corrections, and implement at least two improvements supported by testing evidence.
7. Associate reports, saved routes, and preferences with the signed-in user and prevent other accounts from editing or deleting those personal records.
8. Calculate contribution points and badge eligibility from stored reports, with consistent profile statistics and local rankings and no repeated points for edits.

4. Proposed Solution and Scope

WalkIt will be a user-based Android application whose initial offline prototype stores accounts and hazard reports in SQLite. Signed-in users create reports, view shared local reports, update their own reports and statuses, and delete their own reports after confirmation. Reports are tied to named pathway segments, so the same data supports search, filtering, and route comparison.

Example: a user records an obstruction on a pathway segment. WalkIt then shows that report on every predefined route containing that segment. Marking it Resolved removes it from active-report counts while keeping the record for reference. Deleting it removes the record entirely.

In scope (approved-scope candidate)

- One local installation and one selected demonstration area, with approximately 8-12 predefined pathway segments and 3-5 predefined walking routes (to be finalized in proposal review).
- Complete, owner-restricted CRUD for hazard reports.
- Local registration, login/logout, profile editing, account deletion, badges, leaderboard, saved routes, and personal route preferences.
- Eight functional Activities connected by explicit Intents (registration and login share one Activity).
- Search, category/status filtering, sorting, and route comparison from stored data.
- Fully offline operation with no paid mapping services or backend. Multiple accounts are demonstrated by signing in sequentially on one installation.

Reference segments and routes are seeded once into the database. Demonstration reports are clearly labeled as sample data and are never presented as verified real-world conditions.

How the original WalkIt vision is treated

Original WalkIt feature	CS0011 treatment
Hazard reporting	Core feature with persistent CRUD and input validation.
Accessibility information	Accessibility Barrier category, readable report details, and route-level barrier counts.
Personalized routes	Compare predefined routes for the same origin and destination by distance or active-report counts. No automatic route generation.
Current condition updates	Users update observation date, details, and Active/Resolved status. These are local observations, not community verification.

Original WalkIt feature	CS0011 treatment
User profiles and engagement	Core scope: local accounts, public profile summaries, badges, leaderboard, personal history, saved routes, and preferences.
Interactive map and GPS	Deferred. Named segments and ordered route descriptions provide location context.
Shared reports, community verification, forum	Reports are visible within the same installation only. Cross-device sharing, verification, and discussion are deferred until a backend and moderation exist.
Photo attachments	Optional extension after required scope passes testing.
LGU dashboard, analytics, rewards, advertising, public APIs	Future product development, outside this project. Badges and points carry no monetary value.

SQLite stores data on the device; separate phones do not share accounts, reports, or rankings. The leaderboard is therefore labeled **"Local leaderboard - this installation."** A live community leaderboard and cross-device login would require a backend and a revised, instructor-approved architecture.

5. Core Features

Feature	Planned behavior
Local accounts	Register a unique username and password, sign in, and log out. Only salted password hashes are stored; protected screens are inaccessible after logout.
Personal profiles	Edit display name, preset avatar, and biography; view report totals, points, badges, and join date. Other users see a public read-only summary.
Hazard reporting	Create reports with title, description, category, severity, observation date, and pathway segment. Only the owner may update or delete a report.
My Reports	Review the user's Active and Resolved submissions with search, category/status filters, and sorting.
Route comparison	Compare predefined routes by stored distance, active-report count, or accessibility-barrier count, with clear messages for missing alternatives and unknown conditions.
Saved routes and preferences	Privately save or remove route bookmarks and keep a default comparison preference per account.
Badges and progress	Earn First Step, Pathway Explorer, and Accessibility Advocate badges with visible progress; recalculated after report changes.
Local leaderboard	Rank contributors by derived points, share ranks for ties, and highlight the signed-in user.
Accessible safety information	Category labels, timestamps, and Active/Resolved status shown in readable lists; status never conveyed by color alone.
Account controls	Delete the account with current-password verification and confirmation; this removes owned reports, preferences, bookmarks, and badge awards.

Permissions and personalization

Every report receives its owner ID from the authenticated session. Users may view others' reports and public profile summaries, but only the owner can edit or delete a report, edit a profile, or manage saved

routes and preferences. The interface reflects these permissions, and the data layer enforces them independently. IDs received through Intents are never accepted as proof of identity or ownership. Public profiles never expose passwords, hashes, private saved routes, or preferences.

Contribution points, badges, and leaderboard rules

These are proposed demonstration rules to be reviewed with the instructor.

- A **contribution unit** is one distinct (segment, category) pair represented by at least one retained report owned by the user, Active or Resolved. Each unit earns **10 points**; further reports for the same pair add nothing.
- Editing, toggling status, logging in, or bookmarking a route earns no points. Deleting the last report for a pair removes that unit's points.
- **First Step:** at least one contribution unit. **Pathway Explorer:** retained reports on at least three distinct segments. **Accessibility Advocate:** retained Accessibility Barrier reports on at least three distinct segments.
- Each badge can be held once per user. Badges are current eligibility indicators and are removed if a threshold is lost.
- The leaderboard orders by points, descending. Ties share a competition rank (1, 1, 3), with stable ordering inside ties. Users with zero points show "Not ranked yet."
- Points and badges measure participation, not accuracy or trustworthiness. They do not stop deliberate false reporting; a production system would need moderation and abuse controls.

Worked example: two reports by one user for Obstruction on Segment A earn 10 points. Adding Damaged Pavement on Segment B raises the score to 20. Deleting one of the two Segment A reports leaves 20; deleting the last one drops it to 10.

Route comparison rules

Preference	Ordering rule
Shortest stored route	Sort by stored distance in meters.
Fewest recorded active hazards	Count distinct Active reports on the route's segments; sort by count, then distance.
Fewest recorded accessibility barriers	Count distinct Active Accessibility Barrier reports; sort by that count, then total active reports, then distance.

- Remaining ties are broken by route ID. Resolved reports are excluded from active counts.
- Zero reports are labeled **"No active hazards recorded - conditions may be unknown"**, never as proof of safety.
- If fewer than two alternatives exist, WalkIt shows the available route (or an empty state) and explains that comparison is unavailable.

6. Technical Design

Platform and architecture

The app is built in Android Studio with Kotlin and uses **SQLite via SQLiteOpenHelper**, which the project brief permits. Database operations sit behind a repository/data-access layer and run off the main UI thread. There is no backend or live-map dependency in the first version. Test devices and the final build configuration will be selected during development.

Database design: nine tables

Table	Purpose	Key points
`users`	Account identity and public profile data	Unique lowercase username; display name; avatar key; bio; password hash, salt, and parameters
`hazard_reports`	Owner-linked hazard records	FK to user (cascade) and segment; title 5-80 and description 10-500 chars; severity 1-3; Active/Resolved status; epoch timestamps
`pathway_segments`	Named pathway segments	Unique name and location description
`walking_routes`	Predefined routes	Origin and destination labels; distance in meters (> 0)
`route_segments`	Route membership (many-to-many)	Composite key (route, segment); unique sequence number per route
`badges`	Badge definitions	Unique code; threshold; icon key
`user_badges`	Badge awards	Composite key (user, badge) prevents duplicates
`saved_routes`	Private bookmarks	Composite key (user, route) prevents duplicates
`user_preferences`	Default route preference	One row per user; SHORTEST, FEWEST_HAZARDS, or FEWEST_BARRIERS

- One segment has many reports; routes and segments are linked many-to-many through `route_segments`.
- Deleting a report never deletes its segment or any route. Segment deletion is restricted while reports or route entries exist.
- Deleting an account cascades to its reports, preferences, saved routes, and badge awards in a single transaction; reference data remains intact.
- Points, rank, and report totals are derived by query, not stored, to avoid competing totals.
- Foreign keys are enabled explicitly; all queries use bound parameters; frequently queried fields are indexed; reference data is seeded only at first creation; upgrades must preserve user records.

Local account and session handling

Passwords are stored only as salted hashes from an established hashing implementation, never logged or passed through Intents. Sessions live in memory, so restarting the app requires a new login. Every protected Activity checks the session, every owner-specific query uses the authenticated user ID, and non-launcher Activities are not exported. Login attempts are throttled after repeated failures.

The prototype provides no verified identity, remote password recovery, or cross-device authentication. Its ownership checks demonstrate access control on one installation only, and only synthetic accounts should be used in demonstrations.

7. Screen and Navigation Plan

Activity	Purpose	Intent data
`AuthActivity`	Registration and login; entry point with no active session	Opens `MainActivity` on success; never passes a password

Activity	Purpose	Intent data
`MainActivity`	Report list with search, filter, sort; start report creation or route comparison	`MODE=CREATE`, `REPORT_ID`
`ReportFormActivity`	Validate and save a new or edited report	`MODE`; `REPORT_ID` in edit mode
`ReportDetailActivity`	View a report; edit or confirm deletion (owner only)	`REPORT_ID`
`RouteCompareActivity`	Pick origin/destination and a comparison preference	`ROUTE_ID`, `PREFERENCE`
`RouteDetailActivity`	Ordered segments, distance, counts, related active reports	`ROUTE_ID`, `PREFERENCE`, `REPORT_ID`
`ProfileActivity`	Own profile (edit, preferences, badges, My Reports, Saved Routes, logout, delete) or a public read-only profile	`VIEW_USER_ID` (public only), `SCOPE=MINE`, `ROUTE_ID`
`LeaderboardActivity`	Local rankings and the signed-in user's standing	`VIEW_USER_ID`

Identifiers passed between screens always reload the current SQLite record rather than a stale copy. Missing, invalid, or deleted IDs show an unavailable-record message and a safe route back. My Reports and Saved Routes always use the authenticated user ID, never a supplied profile ID. Low-fidelity wireframes for all eight screens are included in the detailed application proposal.

Validation, feedback, and accessibility

- Forms trim text, enforce length limits, require a valid segment, category, and severity, and reject future observation times. Save is disabled while a write is pending; failed saves keep the user's inputs.
- Registration enforces a unique normalized username, display-name limits, matching passwords, and a proposed 12-128 character password length. Login errors use a generic message.
- Unauthorized changes return "You can only change your own records." Account deletion requires the current password and a confirmation listing what will be removed.
- UI text uses string resources. The design supports text scaling, visible labels, TalkBack descriptions, logical focus order, status text alongside color, and touch targets of at least 48 x 48 dp.

8. Development Plan

The proposed schedule is **eight development weeks after instructor approval**, organized as four two-week sprints using Scrum. Calendar dates will be set once the deadline is announced. Testing, peer review, and documentation happen in every sprint.

Phase	Goal and main deliverables
Proposal and discovery	Submit the proposal, database design, and wireframes; confirm demonstration area and scope; address feedback and obtain instructor approval.
Sprint 1 (Weeks 1-2) Accounts and foundation	Kotlin project, nine-table schema and reference data, account access, sessions, basic profile, owner-linked report creation and list. Demonstrate persistence and separation between two accounts.

Phase	Goal and main deliverables
Sprint 2 (Weeks 3-4) Personal records	Owner-restricted report CRUD, profile editing, My Reports, account deletion, confirmations, search/filter/sort. Test unauthorized edits and account switching.
Sprint 3 (Weeks 5-6) Routes and achievements	Route comparison, private saved routes and preferences, derived points, three badges, local leaderboard. Test duplicate credit, badge removal, ties, and count consistency.
Sprint 4 (Weeks 7-8) Release readiness	Accessibility and user testing, regression tests, documented defect corrections and improvements, APK, documentation, and presentation.

Scrum roles and team allocation

The instructor is a stakeholder and approver, not a Scrum team member. All five members contribute to development and testing; testing is not isolated to one person. Names and final assignments are to be confirmed.

Member	Primary responsibility	Shared responsibility
Member 1	Product Owner: requirements, backlog, proposal coordination	Development, testing, presentation
Member 2	Scrum Master: account access, sessions, integration, navigation	Development, reviews, blocker tracking
Member 3	SQLite schema, owner-restricted CRUD, account deletion	Integration, database testing
Member 4	Profiles, badges/leaderboard UI, resources, accessibility	Form validation, usability testing
Member 5	Route comparison, saved routes, contribution calculations, test coordination	Defect tracking, documentation, release preparation

Events and tracking: Sprint Planning sets a Sprint Goal and Sprint Backlog; Developers hold a 15-minute Daily Scrum; backlog refinement is ongoing; each sprint ends with a Review and a Retrospective with at least one improvement action. A shared board uses To Do, In Progress, Review/Testing, and Done columns.

Feasibility note: the account, badge, and leaderboard features expand the original estimate. During approval the team will confirm that eight weeks is realistic and revise the schedule if not. Profiles, ownership, badges, and the leaderboard are requested scope and will not be silently dropped. Preset avatars, one all-time local leaderboard, and three badges keep the first version bounded.

Initial product backlog

ID	Priority	User story	Minimum acceptance criteria
WI-01	High	As a user, I want an individual account.	Unique registration and login work; no plaintext passwords; logout and restart require authentication.
WI-02	High	As a contributor, I want to record a sidewalk problem.	Inputs validated; owner assigned from the authenticated session.
WI-03	High	As a pedestrian, I want to assess report recency.	Lists and details show observation time, status, category, and segment; filters work.

ID	Priority	User story	Minimum acceptance criteria
WI-04	High	As a pedestrian, I want a route that reflects my preferences.	Predefined alternatives compared by the documented rules, including missing-data states.
WI-05	High	As a user, I want my own profile.	Edits persist; others cannot modify it or see private data.
WI-06	High	As a contributor, I want control over my submissions.	My Reports is owner-filtered; unauthorized update/delete is rejected in the data layer.
WI-07	Medium	As a user, I want quick access to familiar routes.	Save/remove without duplicates; separate lists and preferences per account.
WI-08	High	As a contributor, I want badges that reflect my participation.	Eligibility follows the three rules; no duplicates; removed when thresholds are lost.
WI-09	High	As a contributor, I want to see my local ranking.	Scores agree across profile and leaderboard; ties share rank; edits do not inflate points.
WI-10	High	As a user, I want to remove my account.	Password check and confirmation precede atomic deletion and session clearing.

The Product Owner may reorder stories based on dependencies, feedback, and feasibility. Measurable routing acceptance examples will be agreed during proposal review.

Definition of Done

- Agreed acceptance criteria are satisfied and demonstrated.
- Changes are integrated into the shared app and peer-reviewed where possible.
- Relevant functional checks pass, including permission-denial, invalid-input, and error cases.
- Access controls and any location or photo permissions behave as intended.
- The interface is readable and usable on the agreed test devices.
- Documentation and known limitations are updated; no critical defect blocks the intended flow.

9. Testing and Quality Plan

Testing will use an Android emulator and, if available, a physical Android device. The final report will record the device, Android version, build, date, tester, actual result, and evidence for each case. **All 33 planned test cases below are unexecuted; every initial status is Not Run, and no results are claimed.**

Test area	Cases	What is verified
Report CRUD and persistence	TC-01 to TC-11	Empty state, valid creation, validation and length boundaries, future dates, Intent ID loading, edit, resolve, cancel/confirm delete, persistence after restart.
Search, filter, sort	TC-12, TC-13	Keyword search, combined filters, no-match state, sorting with stable ties.
Route comparison	TC-14, TC-15	Each preference follows its ordering rule; resolved reports excluded; unknown-condition and no-alternative messages.
Robustness	TC-16 to TC-18	Missing or deleted IDs, rotation and rapid Save taps, simulated database write failure.
Accessibility and offline	TC-19, TC-20	Large text and TalkBack across add/edit/delete; core functions with internet disabled.

Test area	Cases	What is verified
Accounts and sessions	TC-21 to TC-23	Registration validation and duplicates, login/logout/restart, no plaintext passwords with unique salts.
Ownership and privacy	TC-24 to TC-26, TC-33	Cross-account edit/delete denial through UI and data layer, private preferences and bookmarks, tampered Intent user IDs.
Points, badges, ranking	TC-27 to TC-29	No extra points for edits or status toggles, badge award and removal, tied ranks (1, 1, 3) and unranked zero-score users.
Account deletion and integrity	TC-30 to TC-32	Verified deletion with cascade, cancelled or wrong-password deletion, transaction rollback on failure.

Defects and improvements

- **Defects:** at least three genuine defects found during development or testing will be documented (linked test, build/device, reproduction steps, expected versus actual behavior, root cause, correction, retest evidence). Records DEF-01 to DEF-03 are open placeholders; the team will not invent failures.
- **Improvements:** at least two changes tied to recorded testing evidence. Candidates are clearer filter visibility with a reset action (IMP-01) and a more prominent unknown-condition message for zero-report routes (IMP-02). If testing does not support them, other observed issues will be chosen, with before/after results recorded.

10. Feasibility, Risks, and Constraints

Risk or constraint	Planned mitigation
Schedule: expanded account features may not fit eight weeks	Confirm feasibility during approval; keep scope bounded (preset avatars, one leaderboard, three badges); return unfinished work to the backlog.
Limited or outdated community data	Label sample reports clearly; show status and observation time; never present zero reports as proof of safety.
Map coverage and connectivity	Avoid live maps and paid services; use named segments, predefined routes, and fully offline operation.
Local-only accounts and rankings	Label the leaderboard as local; document that cross-device sync needs a backend and an approved scope change.
False or duplicate reporting	Points count distinct segment/category pairs only; state that points measure participation, not accuracy.
Privacy and security	Salted password hashes, session-bound ownership checks, bound SQL parameters, synthetic demo accounts, minimal personal data in public reports.
Permissions for location or photos	Request only permissions the chosen functions need; photos and live location remain deferred extensions.

11. Deliverables and Requirement Alignment

CS0011 requirement	Planned evidence
Original, manageable app addressing a clear problem	Hazard management linked to predefined routes in one selected area.
Instructor approval before development	Submitted proposal, feedback record, and approval evidence.
Android Studio and Kotlin	Android Studio project and Kotlin source files.
At least three screens; explicit Intents	Eight Activities passing report, route, profile IDs, form mode, and preference; identity stays session-controlled.
SQLite with complete persistent CRUD	Schema, data-access code, report lifecycle demonstration, persistence tests.
Validation and clear messages	Form rules, confirmations, failure handling, empty states.
Resources, usability, accessibility	Resource files, text-scaling and TalkBack checks, interface review.
Feature beyond CRUD	Accounts, profiles, badges, leaderboard, saved routes, search/filter/sort, route comparison.
At least 12 documented tests	33 planned cases with actual results added during development.
At least 3 defects fixed; 2 evidence-based improvements	Real defect records with retests; two improvements with before/after checks.

Final deliverables: approved proposal and presentation; updated wireframes and database design; prioritized product backlog and sprint records; Kotlin source project; installable APK and setup instructions; test report, defect log, and improvement records; AI-use log and contribution record; updated README; and the final demonstration. The demonstration will show two distinct accounts, owner-restricted CRUD, profiles and bookmarks, badge eligibility, tied leaderboard scores, and how report changes affect route information.

12. Future Enhancements (Out of Scope)

The following belong to the wider WalkIt concept and are not part of this project unless explicitly approved. They are potential directions, not established partnerships, revenue sources, or implemented services.

- LGU and government web dashboard with aggregated reports and infrastructure analytics.
- Cross-device accounts, cloud synchronization, and community-wide leaderboards.
- Live maps and GPS, dynamically generated routes, photo attachments, community verification, discussion, and moderation.
- Community challenges, friend lists, messaging, notifications, and redeemable rewards.
- Local business promotions, analytics services, and licensed APIs for external integrations.

13. Responsible AI Use

AI may assist with proposal organization, concept explanations, test brainstorming, draft interface wording, and debugging suggestions where the instructor permits. Team members remain responsible for understanding, reviewing, implementing, and testing all submitted work.

- Follow the instructor's AI policy and seek clarification before using AI in any restricted part of the assessment.
- Verify technical suggestions against official Android documentation and the app's actual behavior.
- Never upload credentials, private user information, or identifiable reports to AI services; use synthetic examples.
- Keep an AI-use log (date, tool, purpose, prompt summary, output used, team changes, verification).
- Do not fabricate research, test results, defect corrections, citations, or implementation progress.

References

1. Android Developers: Save data using SQLite. https://developer.android.com/training/data-storage/sqlite
2. Android Developers: Intents and intent filters. https://developer.android.com/guide/components/intents-filters
3. Android Developers: Make apps more accessible. https://developer.android.com/guide/topics/ui/accessibility/views/apps-views